

# The MatLang Language Specification

---

## CS4600 — Organization of Programming Languages · Fall 2026 · Version 2.0

MatLang is a small, statically typed language for scalar and matrix arithmetic. It is the source language for the semester compiler project. This document is the normative reference: if your compiler and this document disagree, this document is right.

The language is deliberately small enough to implement completely and precise enough that two students' compilers should accept and reject exactly the same programs.

---

## 1. Design Invariants

Five properties hold throughout. Every rule in this document follows from them, and any extension you add should preserve them.

| Invariant                            | Meaning                                                                                                                                                                           |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Statically typed</b>              | Every expression has exactly one type, computable without running the program.                                                                                                    |
| <b>Dimension-safe</b>                | Matrix shapes are part of the type. Every shape mismatch is a compile-time error.                                                                                                 |
| <b>No implicit conversions</b>       | <code>int</code> and <code>float</code> never convert silently. The single exception is integer <i>literals</i> , defined in §4.3.                                                |
| <b>No declarations without types</b> | There is no inference of variable types; every variable is declared with a complete type.                                                                                         |
| <b>Total static checking</b>         | Only three things can fail at run time: integer division by zero, an out-of-range non-constant index, and inverting a singular matrix. Everything else is caught at compile time. |

---

## 2. Lexical Structure

### 2.1 Source text

A MatLang source file is a sequence of ASCII characters. Source files use the extension `.mat`.

Whitespace — space, tab, carriage return, newline — separates tokens and is otherwise **not significant**. Line structure and indentation carry no meaning.

### 2.2 Comments

```
// line comment, runs to the end of the line
/* block comment, does not nest */
```

Comments are equivalent to whitespace. Block comments do **not** nest: the first `*/` closes the comment. An unterminated block comment is a lexical error.

## 2.3 Tokens

The lexer applies **longest match**: at each position it selects the rule producing the longest lexeme. If two rules tie at that length, the rule listed first below wins.

| Token      | Pattern                                                                             | Notes                             |
|------------|-------------------------------------------------------------------------------------|-----------------------------------|
| FLOAT      | <code>[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?</code><br><code>[0-9]+[eE][+-]?[0-9]+</code> | 3.14, 0.5, 6.02e23, 1e10, 1.5e-10 |
| INTEGER    | <code>[0-9]+</code>                                                                 | 42, 0, 1000                       |
| IDENTIFIER | <code>[A-Za-z_][A-Za-z0-9_]*</code>                                                 | case sensitive                    |
| STRING     | <code>`([^"\\n]</code>                                                              | <code>\["\nt])*`</code>           |

FLOAT is listed before INTEGER only for documentation symmetry; longest match already decides 3.14 correctly on its own. The two rules never tie.

**Why 1e10 needs longest match.** At the 1, INTEGER matches one character and FLOAT matches four. A first-match lexer that tries INTEGER first will tokenize 1e10 as INTEGER(1) IDENTIFIER(e10) — which then fails to parse. Implement longest match, not first match.

## 2.4 Keywords

The following are reserved and may not be used as identifiers:

```
int    float    matrix    if    else    while    print
```

## 2.5 Reserved built-in names

These are not keywords — they are ordinary identifiers that the language pre-binds to built-in functions (§5.7). A program may not declare a variable with any of these names:

```
rows    cols    determinant    inverse    toFloat    toInt
```

## 2.6 Operators and punctuation

| Category            | Tokens               |
|---------------------|----------------------|
| Arithmetic          | <code>+ - * /</code> |
| Transpose (postfix) | <code>'</code>       |
| Assignment          | <code>=</code>       |

| Category                    | Tokens |
|-----------------------------|--------|
| Grouping                    | ( )    |
| Block                       | { }    |
| Matrix literal and indexing | [ ]    |
| Type parameters             | < >    |
| Separators                  | , ;    |

## 2.7 Literals

- **Integer literal** — an `INTEGER` token, optionally preceded by a directly applied unary `-`. Value range is at least 32-bit signed.
- **Float literal** — a `FLOAT` token, optionally preceded by a directly applied unary `-`. Represented as IEEE-754 double.
- **String literal** — a `STRING` token. Strings are **not** a value type; they may appear only as arguments to `print` (§5.8).

**-5 is two tokens.** The lexer emits `MINUS` then `INTEGER(5)`. Negation is handled by the grammar (§3), not by the numeric literal patterns. This is why the patterns in §2.3 have no sign. The phrase "integer literal" in §2.7 and §4.3 refers to a *syntactic form* recognized by the parser, not to a single token.

## 3. Grammar

EBNF. `X*` is zero or more, `X?` is optional, `|` is alternation, quoted text is literal, and `UPPERCASE` names are tokens from §2.3.

```

program      → statement*

statement    → declaration
              | embedded_stmt

embedded_stmt → assignment
              | print_stmt
              | if_stmt
              | while_stmt
              | block

declaration  → type IDENTIFIER '=' expression ';'
              | type IDENTIFIER ';'

type         → 'int'
              | 'float'
              | 'matrix' '<' INTEGER ',' INTEGER '>'

assignment  → IDENTIFIER '=' expression ';'
              | IDENTIFIER '[' expression ',' expression ']' '=' expression ';'

```

```

print_stmt    → 'print' '(' print_arg (',' print_arg)* ')' ';'
print_arg     → STRING | expression

if_stmt       → 'if' '(' expression ')' embedded_stmt
              | 'if' '(' expression ')' embedded_stmt 'else' embedded_stmt

while_stmt    → 'while' '(' expression ')' embedded_stmt

block         → '{' statement* '}'

expression    → additive
additive      → multiplicative (('+' | '-') multiplicative)*
multiplicative → unary (('*' | '/') unary)*
unary         → '-' unary
              | postfix
postfix       → primary ""*
primary       → INTEGER
              | FLOAT
              | IDENTIFIER
              | IDENTIFIER '(' arg_list ')'
              | IDENTIFIER '[' expression ',' expression ']'
              | matrix_literal
              | '(' expression ')'

arg_list      → expression (',' expression)*

matrix_literal → '[' row (',' row)* ']'
row           → expression (',' expression)*

```

### 3.1 Precedence and associativity

Lowest to highest:

| Level | Operators     | Associativity  |
|-------|---------------|----------------|
| 1     | + - (binary)  | left           |
| 2     | * /           | left           |
| 3     | - (unary)     | right          |
| 4     | ' (transpose) | left (postfix) |

So  $-A$  is  $-(A)$ , and  $2 * -3$  is  $2 * (-3)$ . The grammar above encodes this directly; a recursive-descent parser needs no separate precedence table.

### 3.2 Two parsing notes

**Lookahead.** The grammar is LL(1) except at two points, both after an **IDENTIFIER**:

- **statement** — an **IDENTIFIER** begins an assignment, but you must see whether `=` or `[` follows.
- **primary** — an **IDENTIFIER** may be a variable, a call (`(` follows), or an index (`[` follows).

One token of lookahead past the identifier resolves both. A recursive-descent parser peeks; a generated parser handles it automatically.

**Dangling else.** The grammar is ambiguous for nested `if/else`. The resolution is the conventional one: **an `else` binds to the nearest preceding unmatched `if`.**

**Declarations are not embedded statements.** `if (c) int x = 1;` does not parse. A declaration whose scope ends immediately is meaningless, so the grammar forbids it. Write `if (c) { int x = 1; }` if you want a declaration inside a conditional.

## 4. Type System

### 4.1 Types

| Type    | Written                         | Values                                      |
|---------|---------------------------------|---------------------------------------------|
| Integer | <code>int</code>                | signed integers, at least 32-bit            |
| Float   | <code>float</code>              | IEEE-754 double                             |
| Matrix  | <code>matrix&lt;r, c&gt;</code> | an $r \times c$ array of <code>float</code> |

`r` and `c` must be `INTEGER` tokens with value  $\geq 1$ . They are part of the type: `matrix<2, 3>` and `matrix<3, 2>` are different types, and neither is assignable to the other.

**Matrix elements are always `float`.** There is no matrix of `int`. This is why indexing (§5.5) yields `float`, and why a runtime matrix can be a flat array of doubles.

`int` and `float` are the **scalar** types. `matrix<1, 1>` is a matrix, not a scalar; it does not convert to `float`.

### 4.2 Type equality

Two types are equal only if they are identical, dimensions included. There is no subtyping and no coercion.

### 4.3 Context-typed integer literals

There is exactly one place where `int` meets `float`, and it is a property of **literals**, not of expressions.

An **integer literal** (§2.7) appearing directly in a position that requires `float` denotes the corresponding float value.

"Directly" means the literal is the immediate operand of the position. The three positions are:

1. an element of a matrix literal — `[1, 2; 3, 4]`
2. the scalar operand of a `matrix × scalar` or `matrix ÷ scalar` operation — `A * 2`
3. the right-hand side of a declaration or assignment whose target type is `float` — `float x = 3;`

Everywhere else an integer literal has type `int`. In particular, context typing does **not** propagate through arithmetic:

```

float a = 3;          // OK      - rule 3, a == 3.0
float b = -3;         // OK      - rule 3, a signed integer literal is still a
literal
float c = 3 / 2;      // ERROR   - 3/2 is a division, not a literal; its type is int
int    n = 2;
matrix<2, 2> D = A * 2; // OK     - rule 2
matrix<2, 2> E = A * n; // ERROR  - n is a variable, not a literal
matrix<2, 2> F = A * toFloat(n); // OK - explicit conversion

```

This rule exists so that `[1, 2; 3, 4]` and `A * 2` read naturally without weakening the no-implicit-conversions invariant. When you implement it, the type checker passes an *expected type* down into literal nodes; it does not change how any other node is typed.

## 5. Semantic Rules

### 5.1 Declarations and scope

- A declaration introduces a name into the innermost enclosing block, visible from the declaration to the end of that block.
- A **block** opens a new scope. The whole program is the outermost scope.
- Redeclaring a name in the *same* scope is an error. Redeclaring it in an *inner* scope shadows the outer one, which is legal.
- Declaring a name listed in §2.5 is an error.
- Using a name before its declaration is an error.
- A declaration without an initializer (`int x;`) leaves the variable **uninitialized**. Reading an uninitialized variable is not diagnosed; its value is unspecified. Initialize your variables.

### 5.2 Binary operations

`r`, `c`, `k` stand for dimensions. Any combination not listed is a type error.

| Op  | Left        | Right       | Result      | Rule                                               |
|-----|-------------|-------------|-------------|----------------------------------------------------|
| + - | int         | int         | int         | scalar arithmetic                                  |
| + - | float       | float       | float       | scalar arithmetic                                  |
| + - | matrix<r,c> | matrix<r,c> | matrix<r,c> | element-wise; <b>dimensions must be identical</b>  |
| *   | int         | int         | int         | scalar                                             |
| *   | float       | float       | float       | scalar                                             |
| *   | matrix<r,c> | float       | matrix<r,c> | element-wise scaling                               |
| *   | float       | matrix<r,c> | matrix<r,c> | element-wise scaling                               |
| *   | matrix<r,k> | matrix<k,c> | matrix<r,c> | matrix product; <b>inner dimensions must agree</b> |
| /   | int         | int         | int         | truncates toward zero                              |

| Op | Left        | Right | Result      | Rule         |
|----|-------------|-------|-------------|--------------|
| /  | float       | float | float       | IEEE-754     |
| /  | matrix<r,c> | float | matrix<r,c> | element-wise |

Explicitly **not** legal, and each is a useful negative test:

- `int + float` — no conversion, in either direction.
- `matrix + float` — there is no broadcasting.
- `float / matrix` — division by a matrix is not defined.
- `matrix<2,3> + matrix<3,2>` — shapes differ.
- `matrix<2,3> * matrix<2,3>` — inner dimensions 3 and 2 disagree. Element-wise multiplication is not in the language.

### 5.3 Unary minus

| Operand     | Result                              |
|-------------|-------------------------------------|
| int         | int                                 |
| float       | float                               |
| matrix<r,c> | matrix<r,c> (every element negated) |

### 5.4 Transpose

| Operand     | Result      |
|-------------|-------------|
| matrix<r,c> | matrix<c,r> |

Transpose of a scalar is a type error. `A''` has the type and value of `A`.

### 5.5 Indexing

`m[i, j]` where `m` has type `matrix<r, c>`:

- `m` must be a matrix. Indexing a scalar is a type error.
- `i` and `j` must have type `int`.
- The result has type `float`.
- **Indices are 1-based**: valid rows are `1..r`, valid columns are `1..c`.
- If an index is an integer literal, the compiler must reject an out-of-range value at compile time.
- Otherwise the bound is checked at run time.

Indexed assignment `m[i, j] = e`; follows the same rules, and `e` must have type `float`.

### 5.6 Assignment

- The target must be a declared variable or an index into one.
- The type of the right-hand side must **equal** the target's type, dimensions included, subject only to §4.3 rule 3.
- Assignment is a statement, not an expression. `x = y = 0`; does not parse.

## 5.7 Built-in functions

Each takes exactly one argument. Arity and type are checked semantically; the parser does not know these names.

| Signature                                                   | Constraint                     | Notes                                      |
|-------------------------------------------------------------|--------------------------------|--------------------------------------------|
| <code>int rows(matrix&lt;r,c&gt; m)</code>                  | —                              | returns <code>r</code>                     |
| <code>int cols(matrix&lt;r,c&gt; m)</code>                  | —                              | returns <code>c</code>                     |
| <code>float determinant(matrix&lt;n,n&gt; m)</code>         | argument must be <b>square</b> | compile-time shape check                   |
| <code>matrix&lt;n,n&gt; inverse(matrix&lt;n,n&gt; m)</code> | argument must be <b>square</b> | singular matrix is a <i>run-time</i> error |
| <code>float toFloat(int x)</code>                           | —                              | explicit widening                          |
| <code>int toInt(float x)</code>                             | —                              | truncates toward zero                      |

Calling any other identifier is a semantic error ("unknown function"). Because `rows` and `cols` return dimensions that are already in the type, a compiler may constant-fold them; it is not required to.

## 5.8 `print`

`print` takes one or more arguments, each a string literal or an expression, and writes them in order followed by a single newline.

| Argument                       | Output                                                                                        |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| <code>STRING</code>            | the characters, with escapes resolved, verbatim                                               |
| <code>int</code>               | decimal, minus sign if negative                                                               |
| <code>float</code>             | decimal with at least one digit after the point ( <code>2</code> prints as <code>2.0</code> ) |
| <code>matrix&lt;r,c&gt;</code> | <code>r</code> lines, each holding <code>c</code> float values separated by a single space    |

A matrix argument begins on a new line. Adjacent arguments are otherwise concatenated with no separator — put spaces in your string literals.

## 5.9 Conditions

The controlling expression of `if` and `while` must have a **scalar** type, `int` or `float`. A matrix condition is a type error.

Non-zero is true; zero is false. For `float`, both `0.0` and `-0.0` are false; `NaN` is true.

## 6. Execution Semantics

- Statements execute in order. Operands of a binary operator are evaluated **left to right**.
- Matrix values have **value semantics**: `B = A`; copies. Later assignment to `A` does not change `B`.

- Matrix arithmetic uses IEEE-754 double throughout.
- The three run-time errors are: integer division by zero, a non-constant index outside its bound, and **inverse** of a singular matrix. Each must produce a diagnostic naming the source line and terminate the program.

## 7. Diagnostics

Every rejected program must be rejected at the right phase, with a line number. Classify as follows.

| Phase           | Rejects                                             | Example                                                            |
|-----------------|-----------------------------------------------------|--------------------------------------------------------------------|
| <b>Lexical</b>  | input that cannot begin any token                   | <code>int x = 10 @ 20;</code>                                      |
|                 | malformed numeric literal                           | <code>float f = 1.2.3;</code>                                      |
|                 | unterminated string or block comment                | <code>print("oops);</code>                                         |
| <b>Syntax</b>   | a token sequence no grammar rule derives            | <code>int x = ;</code>                                             |
|                 | mismatched grouping                                 | <code>y = (3 + 4;</code>                                           |
|                 | declaration as an embedded statement                | <code>if (c) int x = 1;</code>                                     |
| <b>Semantic</b> | undeclared or redeclared name                       | <code>z = 1;</code> with no <code>z</code>                         |
|                 | type mismatch                                       | <code>int n = 1.5;</code>                                          |
|                 | dimension mismatch                                  | <code>matrix&lt;2,3&gt; P = A * B;</code> with incompatible shapes |
|                 | non-square argument to <b>determinant/inverse</b>   | <code>determinant(matrix&lt;2,3&gt;)</code>                        |
|                 | constant index out of range                         | <code>M[0, 1]</code> — indices are 1-based                         |
|                 | matrix literal whose shape $\neq$ its declared type | <code>matrix&lt;2,2&gt; A = [1, 2, 3];</code>                      |

### 7.1 Matrix literal shape

A matrix literal `[ row ; row ; ... ]` is well-formed only if every row has the same number of elements. Its shape is (number of rows)  $\times$  (elements per row). That shape must equal the declared type exactly.

```
matrix<2, 2> A = [1, 2; 3, 4];      // OK    - 2 x 2
matrix<2, 2> B = [1, 2, 3; 4, 5, 6]; // ERROR - literal is 2 x 3
matrix<2, 2> C = [1, 2; 3];        // ERROR - ragged rows
```

Every element expression must have type **float**, subject to §4.3 rule 1.

## 8. Example Programs

## 8.1 Matrix arithmetic

```
// Basic matrix arithmetic
matrix<2, 2> A = [1, 2; 3, 4];
matrix<2, 2> B = [5, 6; 7, 8];

matrix<2, 2> sum      = A + B;      // element-wise addition
matrix<2, 2> product  = A * B;      // matrix multiplication
matrix<2, 2> scaled   = A * 2.0;    // scalar multiplication

print("A + B =", sum);
print("A * B =", product);
print("A * 2 =", scaled);
```

## 8.2 Linear system solver

```
// Solve Ax = b by matrix inverse
// System: 3x + 2y = 7,  -x + y = 1

matrix<2, 2> A = [3, 2; -1, 1];
matrix<2, 1> b = [7; 1];

float det = determinant(A);

if (det) {
    matrix<2, 2> A_inv = inverse(A);
    matrix<2, 1> x = A_inv * b;

    print("Solution vector x =", x);

    // Verify: Ax should equal b
    matrix<2, 1> check = A * x;
    print("Verification (Ax) =", check);
} else {
    print("System has no unique solution");
}
```

## 8.3 Matrix power by repeated multiplication

```
// Compute A^n
matrix<2, 2> A      = [1, 1; 1, 0]; // Fibonacci matrix
matrix<2, 2> result = [1, 0; 0, 1]; // identity
int n = 5;

while (n) {
    result = result * A;
    n = n - 1;
}
```

```
print("A^5 =", result);
```

## 8.4 Indexing and built-ins

```
// Trace of a square matrix, using 1-based indexing
matrix<3, 3> M = [1, 2, 3; 4, 5, 6; 7, 8, 9];

int i = rows(M);
float trace = 0.0;

while (i) {
    trace = trace + M[i, i];
    i = i - 1;
}

print("trace = ", trace);
```

## 9. Deliberate Omissions

These are not oversights. They are the extension surface for the last phase of the project.

| Not in MatLang                                                                    | Consequence                                                      | Note                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Relational and equality operators                                                 | Conditions test non-zero. Loops count down, as in §8.3 and §8.4. | Adding <code>&lt;</code> and <code>&gt;</code> conflicts with type-parameter syntax — <code>matrix&lt;2, matrix&lt;2,2&gt;&gt;</code> would produce the C++ <code>&gt;&gt;</code> problem. Move dimensions to <code>matrix[2, 2]</code> first. |
| Boolean type and logical operators                                                | No <code>&amp;&amp;</code> , <code>  </code> , <code>!</code> .  |                                                                                                                                                                                                                                                |
| User-defined functions                                                            | Only the six built-ins are callable.                             | The grammar already permits <code>IDENTIFIER '(' arg_list ')'</code> , so this is a semantic-analysis change, not a parser change.                                                                                                             |
| <code>for</code> loops                                                            | Use <code>while</code> .                                         | Pure desugaring.                                                                                                                                                                                                                               |
| Strings as values                                                                 | <code>STRING</code> appears only in <code>print</code> .         |                                                                                                                                                                                                                                                |
| Element-wise matrix product, matrix slicing, <code>matrix&lt;int, r, c&gt;</code> | —                                                                |                                                                                                                                                                                                                                                |
| Runtime-sized matrices                                                            | Dimensions are literals in the type.                             | Dropping this removes the dimension-safe invariant; it is a much larger change than it looks.                                                                                                                                                  |

| Not in MatLang        | Consequence              | Note                                                                                                           |
|-----------------------|--------------------------|----------------------------------------------------------------------------------------------------------------|
| Nested block comments | First <i>*</i> / closes. | Nested comments are not a regular language — they need a depth counter, outside the lexer's regular machinery. |

## Appendix A: Complete Token List

|             |                                                          |
|-------------|----------------------------------------------------------|
| Keywords    | int float matrix if else while print                     |
| Operators   | + - * / ' =                                              |
| Punctuation | ( ) { } [ ] < > , ;                                      |
| INTEGER     | [0-9]+                                                   |
| FLOAT       | [0-9]+\.[0-9]+([eE][+-]?[0-9]+)?   [0-9]+[eE][+-]?[0-9]+ |
| IDENTIFIER  | [A-Za-z_][A-Za-z0-9_]*                                   |
| STRING      | "([^\\"\\n] \\\\"\\n \\\\"\\t)*)"                        |
| Skipped     | [ \t\r\n]+<br>//[^\n]*<br>/\*([^\*] \\*+[^*/])*\*+\/     |

Every example in §8 and every snippet in §4.3, §5.2 and §7.1 was checked against a reference lexer, parser and type checker built from this document.